

Singularity Flow Ways of Working

Drivers, workflows, repositories, examples, and recovery

Singularity Flow runs a change from intake to a published result, checks the exact bytes, and keeps the governed record in Git. The same runtime serves the terminal, GitHub Copilot skills, and the VS Code extension. The surface changes; the lifecycle and authority checks do not.

Validated baseline Repository main 7ecdadb0 on 15 September 2026. Commands in this guide are current at that baseline unless a section is explicitly labelled as a future option.

Guide map

Question	Choices	Decision
Who drives the work	Manual terminal · Copilot assisted · Auto flight	The developer selects the driver
How much workflow	Eligible Story work type and its pinned phase rail	Approved repository configuration
Where the work lives	Repository Story · Local deliverable · SGOS Program	The output and authority boundary
How recovery works	Read state · inspect plan · confirm exact repair	The lifecycle kernel; never chat memory

Use this guide in two ways. Read sections 1–3 for the operating model, then open the worked example that matches your situation. When a command refuses, go directly to section 8 before retrying.

Command names

The executables `sflow` and `singularity-flow` are aliases. Examples use `singularity-flow` when the full name improves clarity and `sflow` in short command sequences. In Copilot, use the `/sf-*` skill shown beside the terminal command. JetBrains hosts may expose the same packaged skill as `/skill:sflow-*`, but the runtime operation is unchanged.

Authority boundary A Copilot reply, a VS Code button, or a successful model call is never an approval. The runtime records a mutation only after the required Git identity, policy, exact hash, and lifecycle checks pass.

1 Choose how the work runs

Manual terminal

Use the terminal when learning the product, diagnosing a refusal, scripting a read-only query, or reviewing every transition. Each mutation is explicit and every command can be inspected with --help.

```
singularity-flow workspace current --json
singularity-flow status --json
singularity-flow nextsteps --json
singularity-flow <command> --help
```

Copilot assisted

This is the normal development path. Copilot drafts artifacts or code; Singularity Flow prepares the governed context, validates output, and records the result. The skill must stop at human decisions and preserve every refusal.

Need	Copilot	Terminal equivalent
Orient	/sf-home or /sf-status	singularity-flow status
Start or resume	/sf-start or /sf-resume	singularity-flow start / resume
Generate a phase	/sf-phase, /sf-requirements, /sf-design, /sf-implement	prepare + phase publish
Submit and decide	/sf-submit, /sf-approve, /sf-reject	submit, approve, reject
Diagnose	/sf-doctor, /sf-recover, /sf-advice	doctor, recover, nextsteps
Inspect grounding	/sf-worldmodel	wm status, wm context, wm ast doctor

Auto bounded flight

Auto is an opt-in repository policy for routine, well-bounded work. Planning may use a model, but it creates only a Plan. Starting requires the Plan's exact digest. The flight does not approve, waive policy, broaden scope, or silently retry a failed model operation.

```
singularity-flow auto plan "Add bounded retry telemetry" --work-type feature --from-branch main --json
singularity-flow auto show-plan <PLAN-ID> --json
singularity-flow auto start <PLAN-ID> --confirm sha256:<PLAN-DIGEST> --json
singularity-flow auto status <FLIGHT-ID> --json
```

If Auto is disabled The refusal is a repository-policy decision. Use the normal Story lifecycle or update Auto eligibility through the approved capability/configuration review path. Do not look for a bypass flag.

2 Choose the lifecycle and storage boundary

Story work types

A Story pins one eligible work type when it starts. That work type defines the ordered phase rail, required artifacts, checks, and approval authorities. Later configuration changes do not silently change an active Story.

Example feature rail at this baseline

```
intake → requirements → design → implementation-spec → implementation → verification → conformance
```

Use workflow list --for-start before intake. The result contains only workflows the current authority permits for new work.

```
singularity-flow workflow list --for-start
singularity-flow progress <WORK-ID> --markdown
```

Repository Story

Use a Repository Story for source changes that will land in Git. The Story owns a branch, a managed working copy, pinned configuration, generated artifacts, approvals, and publication receipts.

Local signed deliverable

Use local mode when the output must be audited but no product repository exists. It captures only the paths you name, keeps a private per-Story ledger, and can publish an offline-auditable signed ZIP to an approved local directory.

SGOS Program

Use an SGOS Program for an immutable compiled execution graph. Program validation, explanation, simulation, and what-if analysis are read-only. Program approval publishes a configuration review proposal; it does not grant authority until the normal review path places it on approved sflow/config.

3 Worked example for an assisted Feature Story

Scenario PAY-1234 adds partial refunds to a payments API. A valid partial amount must create one auditable refund. The repository uses the feature workflow and main is the approved base branch.

Step 1 Verify the selected workspace

Begin with a read. The returned repositoryPath is the only valid command boundary. If the selected workspace or repository is wrong, change it before starting the Story.

```
singularity-flow workspace current --json  
singularity-flow workflow list --for-start
```

Step 2 Start the Story

```
singularity-flow start PAY-1234 \  
--work-type feature \  
--title "Add partial refund support" \  
--description "Allow an operator to partially refund a captured payment." \  
--acceptance-criteria "A valid partial amount creates one auditable refund." \  
--from-branch main --fetch
```

From Copilot, run /sf-start. The skill collects the same fields, shows the repository and work type, and calls the same runtime command after the required confirmation.

Expected durable result One Story-opening commit pins the source revision, work type, capability, configuration, and return locator. No phase is approved by starting the Story.

Step 3 Run the artifact phase loop

Use the phase name returned by progress or nextsteps. The example below uses requirements.

```
singularity-flow prepare requirements  
# Complete the generated requirements artifact  
singularity-flow phase draft-check requirements --json  
singularity-flow phase publish requirements --authored governed-agent --channel copilot-host  
singularity-flow submit requirements --work-id PAY-1234  
singularity-flow approve requirements --work-id PAY-1234 --fetch  
singularity-flow nextsteps PAY-1234 --json
```

Operation	What it proves	What it does not prove
draft-check	The current draft meets deterministic content rules	Publication or approval

Operation	What it proves	What it does not prove
phase publish	Generation N binds exact artifact bytes and authorship	Reviewer acceptance
submit	A review packet binds the published generation	Approval authority
approve	The current Git identity decided on the exact submitted hash	Independent review when self-approved

Step 4 Open the implementation boundary before editing

Implementation is different because application source and tests may change. Open the generation intent before the first source edit, then author the change and acceptance-mapped tests.

```
singularity-flow prepare implementation
singularity-flow phase begin implementation --json
# Edit source and tests
singularity-flow phase draft-check implementation --json
singularity-flow phase publish implementation --authored governed-agent --channel copilot-host \
--change-origin copilot --change-origin human
singularity-flow submit implementation --work-id PAY-1234
```

Important Do not add or repair workflow.yml inside an active implementation generation. Configuration is a protected authority. Change it through Configuration Center or a configuration review, then start new work from the refreshed authority.

Step 5 Complete the Story

Repeat the publish, submit, and approval loop for each phase shown by progress. Finalization runs the terminal gate and prepares the governed landing action only after every required phase is complete.

```
singularity-flow progress PAY-1234 --markdown
singularity-flow status PAY-1234 --submission-readiness --json
singularity-flow finalize --json
```

4 Worked example for capability and workspace onboarding

Scenario A payments API already exists in Git. The organisation map is owned by a separate platform repository. A new laptop must discover the existing authority instead of creating a duplicate capability.

Step 1 Inspect the repository first

```
singularity-flow capability inspect-repository \
"https://git.example.corp/acme/payments-api.git" \
--search-known --include-proposals --refresh --json
```

The inspection distinguishes already mapped, pending proposal, new, ambiguous, and unavailable states. If it reports already mapped, attach that capability to a workspace. Do not create another map entry.

Step 2 Propose and activate only when the repository is new

```
singularity-flow capability map payments-api \
--lead "https://git.example.corp/acme/platform.git" \
--name "Payments API" --kind delivery \
--repository "https://git.example.corp/acme/payments-api.git" --json

singularity-flow capability proposal <REVIEW-BRANCH> \
--lead "https://git.example.corp/acme/platform.git" --json

singularity-flow capability activate <REVIEW-BRANCH> \
--lead "https://git.example.corp/acme/platform.git" \
--confirm <FULL-PROPOSAL-COMMIT> --json
```

Mapping creates a review branch against sflow/config. Activation binds the reviewed full commit. If the configuration merge succeeds but state projection is pending, run the exact capability publish action returned by the result; do not repeat the merge.

Step 3 Create a recoverable workspace

```
singularity-flow workspace prepare \  
"https://git.example.corp/acme/platform.git" \  
--id payments-work --capability payments-api \  
--lead-capability payments-api \  
--base "/approved/workspaces" --initialize --json  
  
singularity-flow workspace bootstrap resume <BOOTSTRAP-ID> \  
--confirm payments-work --json  
singularity-flow workspace use payments-work --repository payments-api --json  
singularity-flow workspace current --json
```

prepare records a resumable plan and checks destinations, branches, disk, Git, and network without materialising the workspace. resume rechecks the same plan before cloning. A failed clone remains attached to the bootstrap ID, so a corrected proxy or credential can retry the same operation.

Attach an existing capability to an existing workspace

```
singularity-flow workspace attach-capability \  
"/approved/workspaces/payments-work" payments-api --dry-run --json  
singularity-flow workspace attach-capability \  
"/approved/workspaces/payments-work" payments-api \  
--confirm-plan <PLAN-ID> --json
```

Fail-safe rule A dry-run plan is bound to the observed map and workspace state. If either changes, confirmation returns stale-plan. Re-run the preview and review the new plan; never reuse the old ID.

5 Worked example for a new laptop or old workspace

Existing clone on a new laptop

Onboarding attaches the checkout to a verified authority pin. It does not clone, build AST, build a world model, or invoke a model. If more than one remote can own authority, name the remote explicitly.

```
cd /approved/workspaces/payments-api  
sflow onboard . --remote origin --json  
singularity-flow doctor --json
```

Return to a published Story

return uses the published Story locator. Preview first; apply requires the exact Work ID and a clean worktree. It fast-forwards only the local Story branch and reconstructs the session.

```
singularity-flow return PAY-1234 --json  
singularity-flow return PAY-1234 --apply --confirm PAY-1234 --json
```

Refresh packaged configuration

Use refresh-configuration when the repository authority is valid but packaged workflows, templates, prompts, or agents are older. Preview conflict choices, then confirm the current plan.

```
singularity-flow workspace refresh-configuration payments-work --dry-run --json  
singularity-flow workspace refresh-configuration payments-work \  
--confirm-plan <CFGP-ID> --json
```

Reinitialize an older format

Use reinitialize for the complete safe upgrade path. It delegates approved configuration updates to the same three-way refresh, verifies capability authority locators, and checks durable schema readability. Existing Story snapshots remain pinned and immutable history is not rewritten.

```
singularity-flow workspace reinitialize payments-work --dry-run --json
singularity-flow workspace reinitialize payments-work \
--confirm-plan <PLAN-ID> --json
```

Configuration scope Updated sflow/config and its state projection govern new Stories. An active Story keeps the configuration snapshot it started with unless a separately governed lifecycle explicitly opens a compatible new generation.

6 Worked example for world model and AST intelligence

World model choices

Need	Command	Model use
Fast inventory	singularity-flow wm light	None
Private rehearsal	singularity-flow wm light --local	None
Narrative build	singularity-flow wm build --depth quick standard deep	Configured provider
Read status	singularity-flow wm status --json	None
Compose phase context	singularity-flow wm compose --phase <PHASE> --work-id <ID>	None

The reusable model is keyed to the repository source and approved scope, not to a Story name. A verified model published on the configured state branch can serve another Story or laptop at the same source revision. Lifecycle-only changes do not justify rebuilding it.

Build deterministic inventory without a model

```
singularity-flow wm light --views architecture,development,testing
singularity-flow wm check
singularity-flow wm status --json
```

Build a deeper narrative when explicitly required

```
singularity-flow wm build --depth deep --parallel --workers 4
singularity-flow wm status --json
```

wm build is the model-backed operation. Discovery checkpoints are reusable, and a failed publication retains validated recovery material where policy permits. Do not rebuild merely because a new Story starts; check status first.

Use optional AST intelligence

```
singularity-flow wm ast preference set auto --json
singularity-flow wm ast doctor --json
singularity-flow wm ast build --all --json
singularity-flow wm ast query --predicate symbol --value PaymentService \
--paths src --max-facts 50 --max-output-bytes 32768 --json
singularity-flow wm ast preference set off --json
```

AST is optional auto uses an available reviewed provider; off is a valid disabled result. Missing packs, unsupported languages, incomplete project binding, and cache failures remain advisory. Ordinary Copilot file access and lifecycle recovery continue without AST.

7 Worked example for local signed delivery

Scenario A reviewed SQL directory must be converted into a deliverable before a target repository exists. The input is internal, and the bundle must be auditable offline.

```
sflow local start conversion \  
  --intent "Convert reviewed SQL into a PySpark batch job" \  
  --input /approved/input/sql --classification internal --json  
sflow local status --story <LOC-ID> --json  
# Produce output only in the reviewed output directory  
sflow local freeze --story <LOC-ID> --output /approved/output/pyspark --json  
sflow local verify --story <LOC-ID> --candidate <SHA256> --json  
sflow local signer-create --story <LOC-ID> --signer local-owner --json  
sflow local trust-export --story <LOC-ID> --signer local-owner \  
  --out /approved/trust/local-owner.pem --json  
sflow local review --story <LOC-ID> --candidate <SHA256> --signer local-owner --json  
sflow local publish --story <LOC-ID> --candidate <SHA256> \  
  --signer local-owner --destination /approved/exports --json  
sflow local audit --bundle /approved/exports/<BUNDLE>.zip \  
  --trust-key /approved/trust/local-owner.pem --signer local-owner --offline --json
```

Local mode proves the exact captured inputs, frozen outputs, verification record, signatures, and bundle bytes. It does not claim semantic correctness, current enterprise authorization, deployment, or a successful rerun. Repository adoption remains a separate governed action.

8 Recover from refusals and interruptions

A refusal is a safe outcome. Read its stable code and remediation plan, verify the selected repository and Story, and run the smallest read-only diagnostic. Repeating the same failed mutation without changing its preconditions cannot help.

General recovery sequence

1. Confirm the current workspace, repository, Story, phase, and branch.
2. Run doctor and the phase-specific recover preview. Save the returned plan ID or hash.
3. Review every path and effect. Apply only an automatic plan with its exact confirmation.
4. Re-read status or progress. Retry publication only if the blocker fingerprint changed.

```
singularity-flow workspace current --json  
singularity-flow status <WORK-ID> --json  
singularity-flow doctor --json  
singularity-flow recover <WORK-ID> --phase <PHASE> --json  
singularity-flow recover <WORK-ID> --phase <PHASE> \  
  --apply --confirm <PLAN-HASH> --json
```

Common blocker patterns

Blocker	Safe response	Do not
Draft contains an unresolved placeholder	Run phase draft-check, complete the authored content, then publish	Submit or approve an incomplete draft
Generation intent consumed and bytes changed	Preview phase rollover; run only the returned exact confirmation	Guess or reuse an old change-set digest
Governed commit exists but push failed	Run sync so the exact recorded commit is retried	Create a replacement commit or push current HEAD
Protected workflow/configuration path changed	Restore the Story edit; use Configuration Center or a configuration proposal	Bypass protected-path policy
Plan is stale	Re-run the dry-run/preview and review its new plan	Reuse the previous plan ID
World-model grounding requested	Run wm status, then the exact wm ensure or compose action returned	Rebuild a reusable model just because the Story ID changed

Interrupted commit or push

Recovery uses the durable publication journal. Before a governed commit, it restores only the journaled governed preimage and retains interrupted bytes in a rescue directory. After a commit, sync pushes the exact recorded commit. It never pushes an arbitrary later HEAD.

```
singularity-flow doctor --json  
singularity-flow sync
```

Changed published generation

```
singularity-flow phase rollover implementation --json  
# Review the current digest and effects  
singularity-flow phase rollover implementation --confirm <CURRENT-DIGEST>
```

Stop condition If recover returns the same blocker and plan ID after the prescribed action, stop. The remaining step needs a human policy/content decision or an external condition such as Git access; repeated retries only add delay.

9 Switch Stories and continue on another machine

Switch within one workspace

Multiple Stories may be active, but one session selection is current. List candidates, then resume or attach the exact Work ID. Dirty work is never discarded automatically.

```
singularity-flow session candidates --json  
singularity-flow resume PAY-5678 --fetch  
singularity-flow session attach PAY-5678 --json  
singularity-flow workspace use payments-work \  
--repository payments-api --story PAY-5678 --json
```

Continue on another machine

```
singularity-flow return PAY-1234 --json  
singularity-flow return PAY-1234 --apply --confirm PAY-1234 --json
```

The return locator contains portable repository and lifecycle identities, not credentials or machine paths. Git Credential Manager, SSH, or the organisation's approved Git setup supplies authentication outside Singularity Flow.

10 Daily checklist and glossary

Before starting work

- Run workspace current and confirm repositoryPath, repository ID, and active Story.
- Run workflow list --for-start for a new Story; do not assume a workflow from another workspace.
- Inspect a repository before capability mapping; reuse an existing authority when one is found.
- Check wm status before building. Reuse a fresh repository model.

Before publishing a phase

- Run phase draft-check and resolve every placeholder or structured finding.
- For implementation, open phase begin before source changes and include acceptance-mapped tests.
- Keep workflow.yml, agents, templates, and other protected process files out of Story implementation changes.
- Review authorship, model usage, and change-origin metadata before phase publish.

Before approval or finalization

- Open the exact artifact and review packet; do not approve from a chat summary.
- Answer every required checklist article explicitly.
- Treat self-approval warnings as evidence about review independence, not as a hidden pass.
- Run status --submission-readiness and progress before finalize.

Glossary

Term	Meaning
Capability	A governed description of what the organisation builds and which repositories deliver it
Workspace	A machine-local selection of governed repositories, capabilities, and active work
Story	One governed change with a pinned workflow, source revision, branch, artifacts, and lifecycle state
Phase	One ordered lifecycle step with required inputs, outputs, checks, and approvals
Generation	A numbered, exact publication of a phase artifact or implementation change set
Review packet	The hash-bound material submitted for an approval decision
Receipt	A durable record of an operation and the exact bytes or identities it bound
Refusal plan	A structured failed result with stable code, safe diagnostic steps, and retry conditions
State branch	The governed projection and reusable runtime evidence shared without changing application main
World model	Repository grounding tied to source and approved scope; deterministic inventory or optional model-authored narrative
AST intelligence	Optional bounded structural facts from reviewed language providers and caches

Authoritative references

The running code and approved repository configuration remain authoritative. For more detail, use the repository files below and the exact --help output from the installed build.

- README.md and HELP.md for command entry points and complete syntax
- docs/GOLDEN-JOURNEY.md for the developer journey and cross-machine return
- docs/FAST-ONBOARDING-AND-GIT-PERFORMANCE.md for onboarding and safe Git behavior
- docs/VS-CODE.md for visual surfaces and confirmation boundaries
- docs/topics/recovery.md for interruption and refusal recovery
- docs/topics/world-model.md and docs/topics/ast-intelligence.md for grounding
- docs/LOCAL-SIGNED-DELIVERABLES.md for repository-free delivery

Everyday path Select the workspace, start or resume the Story, follow nextsteps, draft through /sf-* or the terminal, run draft-check, publish, submit, review the exact packet, approve, and finalize. When anything refuses, preserve the state and follow the returned recovery plan.